EXPERIMENT 13

LIMITATIONS OF LOW-CODE PLATFORMS

Student Name: Ajayshankar Giridharan **Roll Number:** 21401007

1. Aim

To evaluate the strengths and limitations of low-code platforms using Zoho Creator and the loan application as a case study.

2. Objectives

• Identify technical and organizational limitations.
• Assess implications for performance, customization, portability, governance, and security.
• Separate suitable low-code use cases from cases that may need pro-code extensions.
• Recommend mitigation strategies.

3. Platform and Application Context

Platform: Zoho Creator
Application: Loan Application & Pre-Screening Management
Form: Loan_Application_Form

Student Roll No.: 21401007

4. Theory

Low-code platforms accelerate application delivery through visual builders, managed hosting, and reusable components. They can still impose limits around highly specialized logic, performance at scale, platform-specific skills, vendor dependence, and control over infrastructure. Suitability depends on complexity, risk, data sensitivity, integration needs, and expected scale.

5. Procedure

1. Use the loan application as a case study and identify its form, report, dashboard, Deluge workflows, connection, and roles.
2. Evaluate each limitation category in the comparison table below.
3. For each limitation, identify a practical mitigation such as testing, API documentation, monitoring, access reviews, or a pro-code extension.
4. Decide which features are appropriate for native low-code and which may require external services or professional code.
5. Summarize the trade-off between speed of development and long-term control.

6. Observations / Test Results

Area	Observation	Status / limitation
Customization	Complex or unusual logic may exceed built-in visual options.	Use Deluge or a controlled external service where justified.
Performance / scale	Large datasets, complex reports, or high workflow volume can affect responsiveness.	Measure real workloads; optimize reports and test at expected scale.
Portability	Platform-specific schema and scripts may create migration costs.	Document data models, APIs, export paths, and dependencies.
Governance / security	Access mistakes and unclear ownership can expose records or create maintenance risk.	Use least privilege, audit reviews, backups, and lifecycle controls.
Vendor dependence	Platform service limits and pricing/feature changes can affect plans.	Review service limits, data export, and continuity requirements.

7. Result

The evaluation identifies low-code as useful for structured business workflows and rapid delivery, while complex, highly scalable, or platform-independent requirements may justify a hybrid or pro-code approach.

8. Conclusion

Low-code should be selected with an explicit understanding of its constraints. These limitations are manageable when the application is tested, governed, documented, and extended only where necessary.

Decision guide

Good fit: forms, routine approvals, departmental dashboards, internal workflow automation.
Evaluate carefully: high-volume reporting, complex integrations, sensitive data, strict latency requirements.
Consider pro-code/hybrid: specialized algorithms, heavy compute, custom infrastructure requirements, or systems requiring

extensive portability.

9. Evidence

The figure below is a generated explanatory schematic based on the documented experiment context. It is clearly labeled illustrative and is not a captured screenshot of a live deployment or a claim that untested actions succeeded.

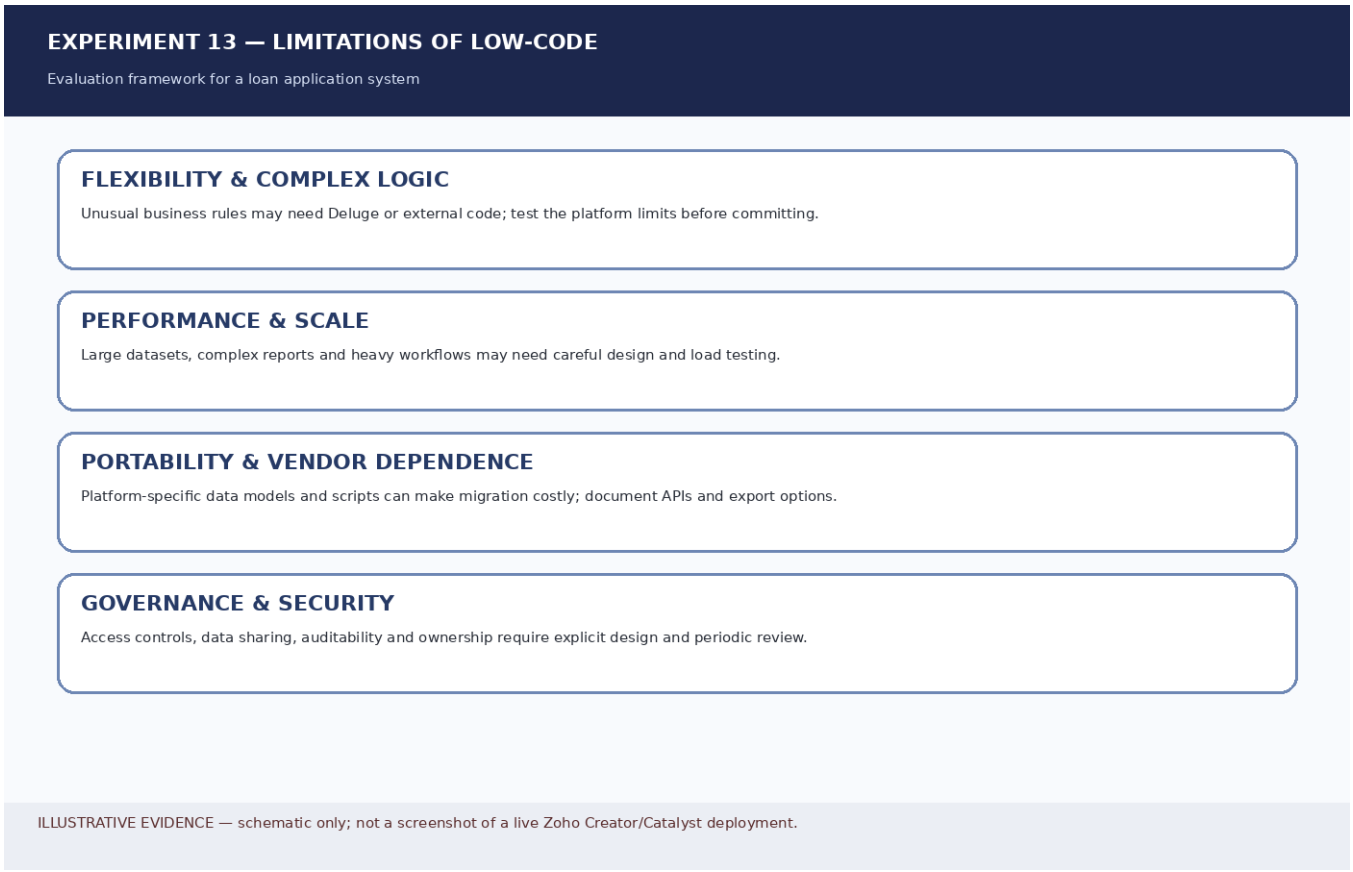


Figure 13.1 — Illustrative evaluation framework for common low-code limitations.

Submission note: Where the experiment requires live configuration evidence, replace or supplement this schematic with your own screenshot from Zoho Creator/Catalyst after completing the corresponding step. Do not submit the schematic as a real system screenshot.